

// Day 17 of 30

VISUAL EXPLAINER

# JWT Authentication — Under the Hood

→ Swipe through all 6 slides



**Shubham Bhati**

[linkedin.com/in/bhatishubham](https://www.linkedin.com/in/bhatishubham)

1/8

# What is JWT

JSON Web Token — your digital ID card

JWT = three parts separated by dots:

eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXZWJ0In0.

|

Header

(algo)

|

Payload

(data)

|

Signature

(verification)

💡 JWT is NOT encrypted. It's SIGNED. Anyone can read it.



**Shubham Bhati**

[linkedin.com/in/bhatishubham](https://www.linkedin.com/in/bhatishubham)

## Header

Algorithm + type

```
{  
  "alg": "HS256",  
  "typ": "JWT"  
}
```

- Base64 encoded (not encrypted!)
- Tells server which algorithm was used to sign

💡 Header is public. Never put secrets here.



**Shubham Bhati**

[linkedin.com/in/bhatishubham](https://www.linkedin.com/in/bhatishubham)

## Payload

Your actual data (claims)

```
{  
  "sub": "user123",  
  "name": "Shubham",  
  "role": "ADMIN",  
  "iat": 1712345678,  
  "exp": 1712349278  
}
```

- Also Base64 encoded
- Anyone can decode and read
- NEVER put passwords here

💡 exp = expiry time. Always set it. Always.



**Shubham Bhati**

[linkedin.com/in/bhatishubham](https://www.linkedin.com/in/bhatishubham)

## Signature

The security part

```
HMACSHA256(  
  base64(header) + "." + base64(payload),  
  your-secret-key  
)
```

- If anyone changes the payload
- Signature won't match
- Server rejects the token

This is how JWT prevents tampering.

💡 Secret key stays on server. NEVER expose it.



**Shubham Bhati**

[linkedin.com/in/bhatishubham](https://www.linkedin.com/in/bhatishubham)

## Login Flow

How it works end-to-end

1. User sends: `POST /login`  
`{email, password}`
2. Server validates credentials
3. Server creates JWT with user data
4. Server sends JWT to client
5. Client stores JWT (`localStorage`)
6. Every next request:  
Authorization: Bearer `eyJhbG...`

💡 Server never stores sessions. JWT is stateless.



## Spring Security + JWT

How we implement it

OncePerRequestFilter:

- Extract token from header
- Validate signature
- Check expiry
- Set SecurityContext

```
@PreAuthorize("hasRole('ADMIN')")
public ResponseEntity adminOnly() {
    // Only ADMIN role can access
}
```

💡 Spring Security + JWT = industry standard auth



**Shubham Bhati**

[linkedin.com/in/bhatishubham](https://linkedin.com/in/bhatishubham)

done = true

**Follow for more!**

Like · Comment · Repost

**Shubham Bhati**

[linkedin.com/in/bhatishubham](https://www.linkedin.com/in/bhatishubham)



**Shubham Bhati**

[linkedin.com/in/bhatishubham](https://www.linkedin.com/in/bhatishubham)